

# TruthLens — Week 2: Design & Planning

cseku\_26\_wpl\_truthlens — branch 0.2

Evidence-based AI analysis platform for text, image, and video misinformation and deepfake detection.

## 1. UI wireframes (desktop)

Six core screens cover the MVP: Home, Text analysis, Image analysis, Video analysis, Results (shared across all three types), and History. All screens share a persistent top nav (logo, section links) and a wide two-column layout on desktop — a primary action column plus a supporting sidebar (“How it works” / “What we check” / “Evidence”) that fills the extra horizontal space rather than leaving it empty. Home uses a single paste-or-drop input with auto type detection, rather than forcing a format choice up front.

Figure 1.1 — Home

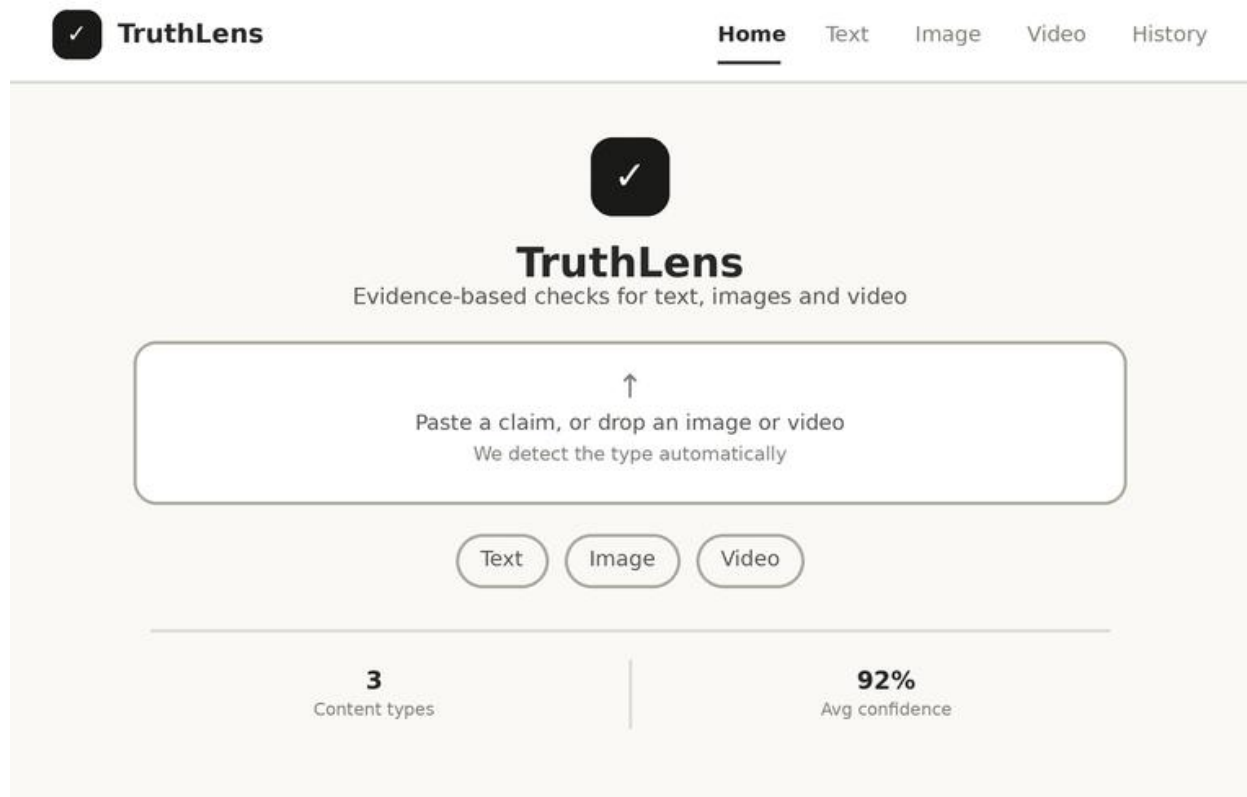


Figure 1.1: Home screen — persistent nav, a single paste-or-drop zone that auto-detects text, image, or video, format chips for explicit selection, and content-type / confidence stats below.

Figure 1.2a — Text analysis

✓

TruthLens

HomeTextImageVideoHistory

T

Check a claim

Paste text and we'll verify it against trusted sources

↑

Type or paste a claim to verify

"The Great Wall is visible from space"

0 / 500 · Plain text only

Check claim →

Cancel

How it works

1

We retrieve related evidence from FEVER

2

A verification model scores support / refute

3

You get a confidence score with sources

Figure 1.2a: Text analysis — claim input on the left, a “How it works” panel on the right.

Figure 1.2b — Image analysis

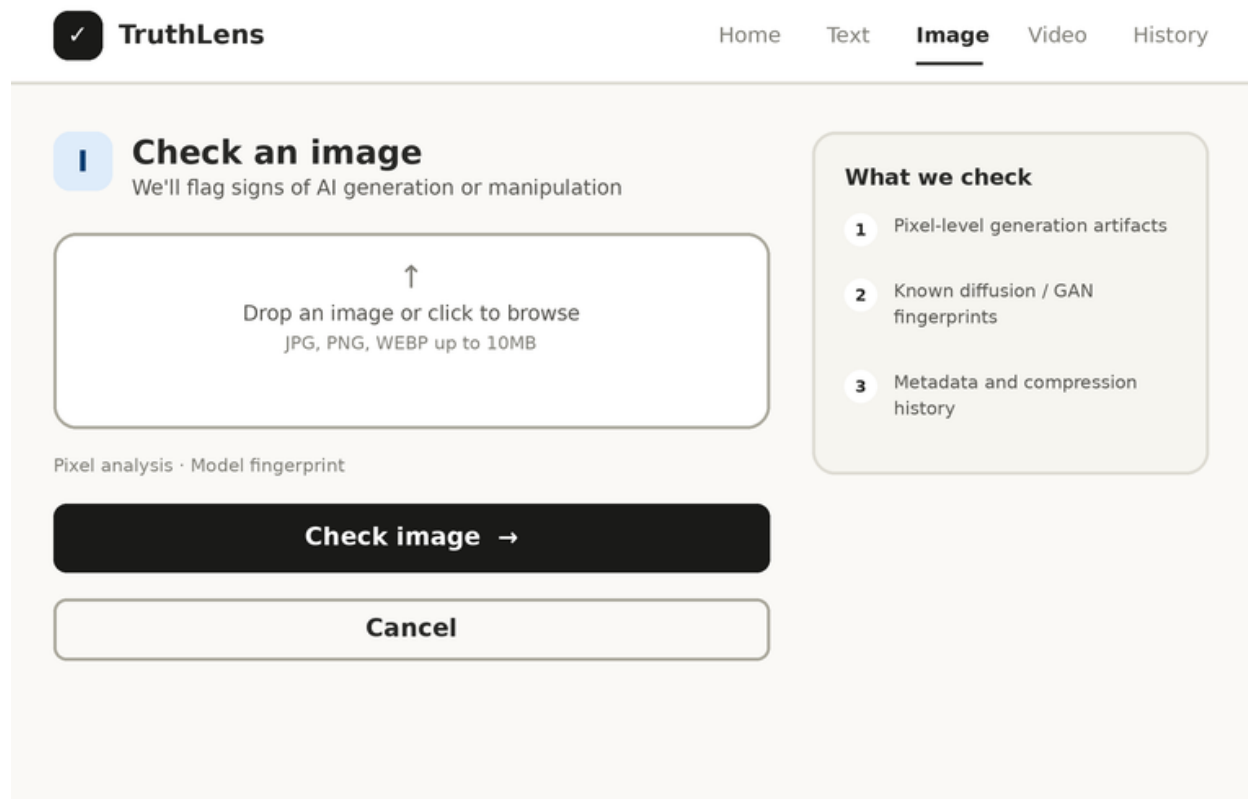


Figure 1.2b: Image analysis — drop zone on the left, “What we check” panel on the right.

Figure 1.2c — Video analysis

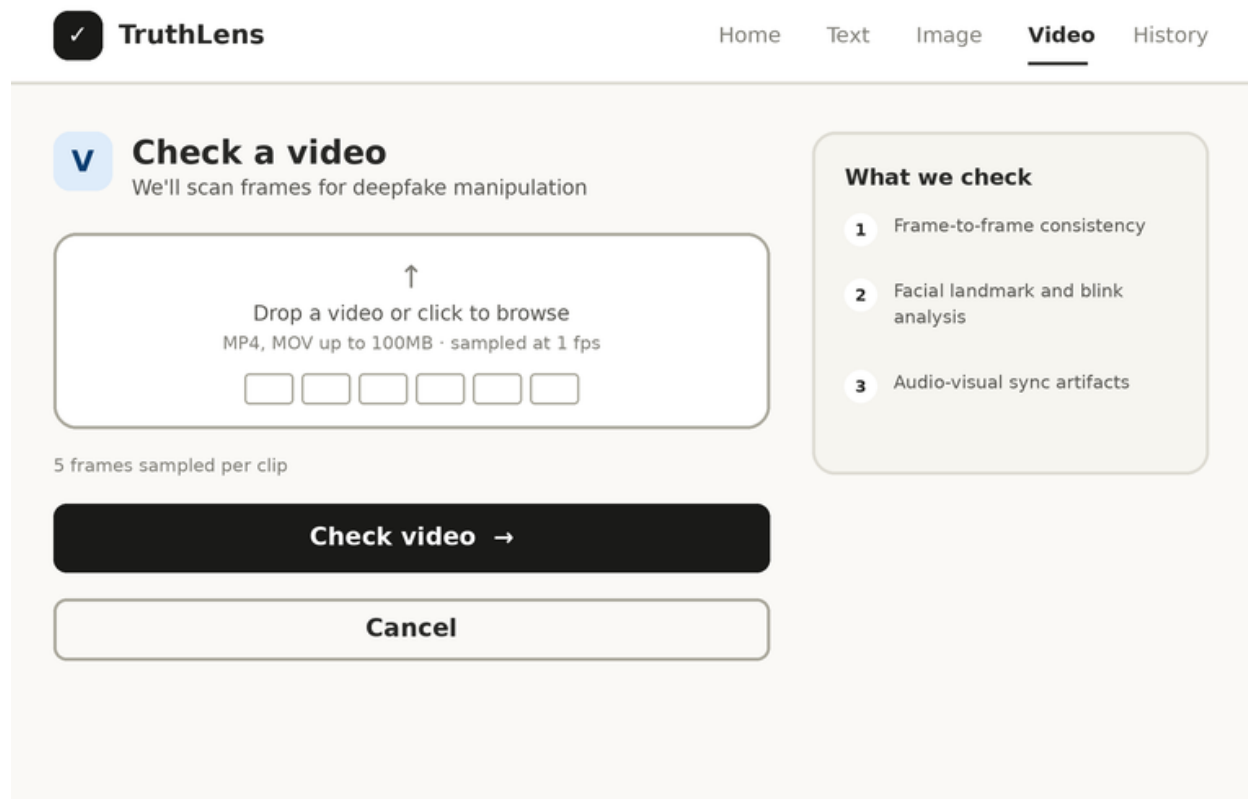


Figure 1.2c: Video analysis — drop zone with a sampled-frame preview strip, “What we check” panel on the right.

Figure 1.3 — Results (shared screen)

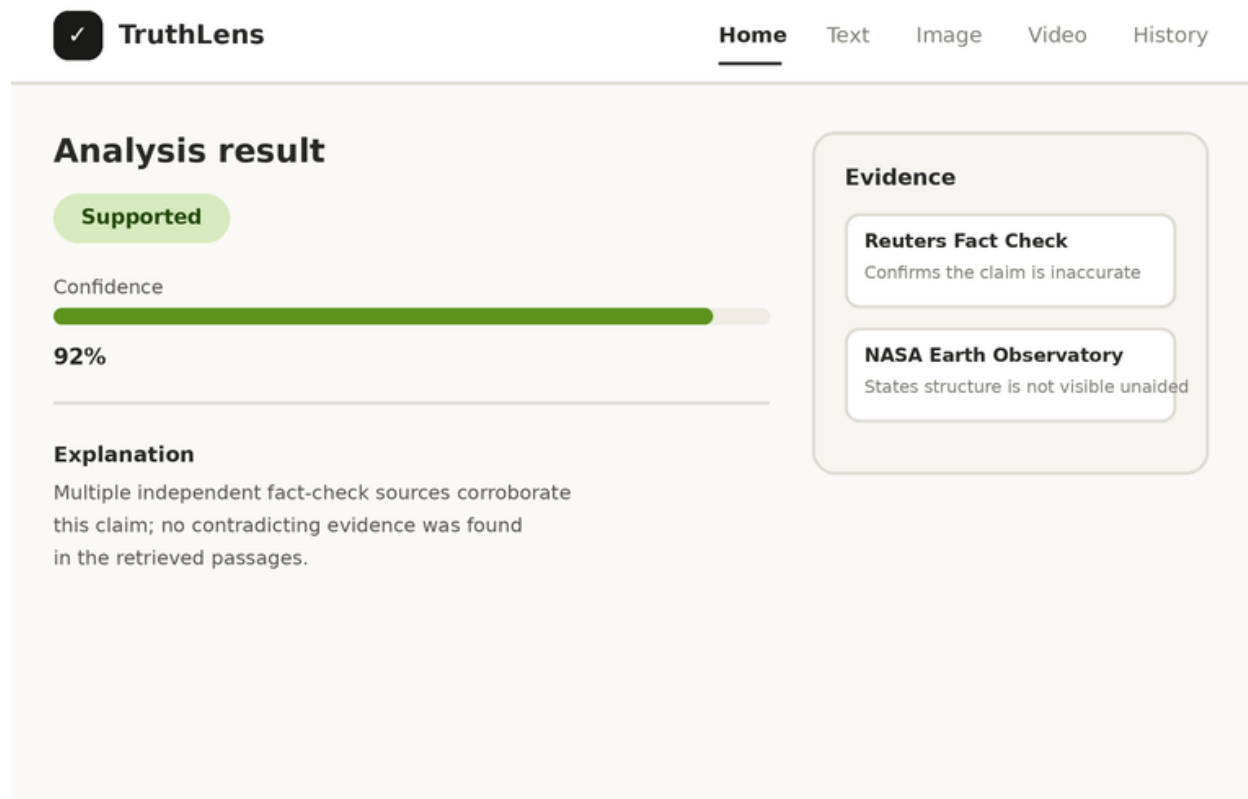


Figure 1.3: Shared result screen — assessment, confidence and explanation on the left; supporting evidence cards on the right.  
Reused across text, image, and video.

## Figure 1.4 — History



## Analysis history

| Type  | Claim / file                    | Date   | Confidence | Result       |
|-------|---------------------------------|--------|------------|--------------|
| Text  | "Great Wall visible from space" | 24 Aug | 92%        | Supported    |
| Image | profile_edited.jpg              | 24 Aug | 87%        | AI-generated |
| Video | interview_clip.mp4              | 23 Aug | 82%        | Manipulated  |
| Text  | "Coffee stunts growth"          | 22 Aug | 78%        | Refuted      |

Figure 1.4: History as a proper desktop data table — type, claim/file, date, confidence, and result at a glance.

## 2. System architecture

React frontend talks to a FastAPI backend over REST. The backend fans out to three independent ML services — text, image, and video — each grounded in its own dataset (FEVER, GenImage, FaceForensics++). All three feed into a shared result engine that returns assessment, confidence, and explanation.

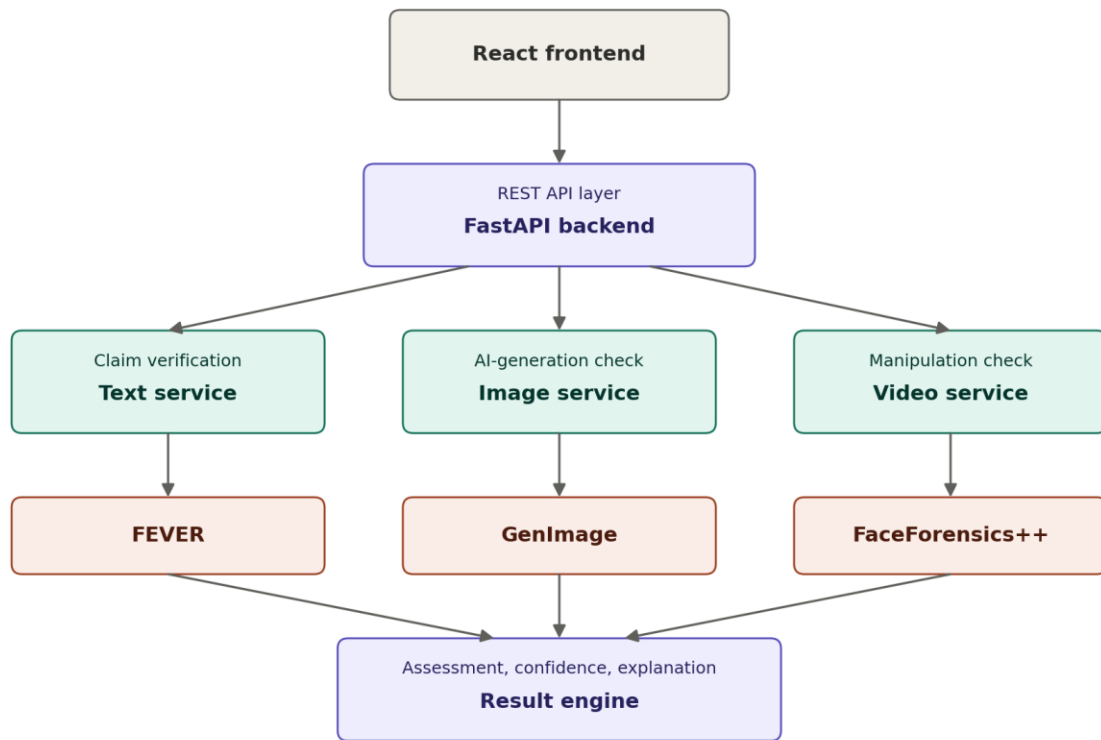


Figure 2.1: TruthLens system architecture — frontend, backend, three ML services, and the shared result engine.

Keeping the three services independent keeps the codebase easier to maintain and test in isolation.

### 3. Data flow

All three input types follow the same shape: submit -> POST to backend -> preprocessing -> model -> confidence + explanation back to frontend. Only the preprocessing and model step differ per type.

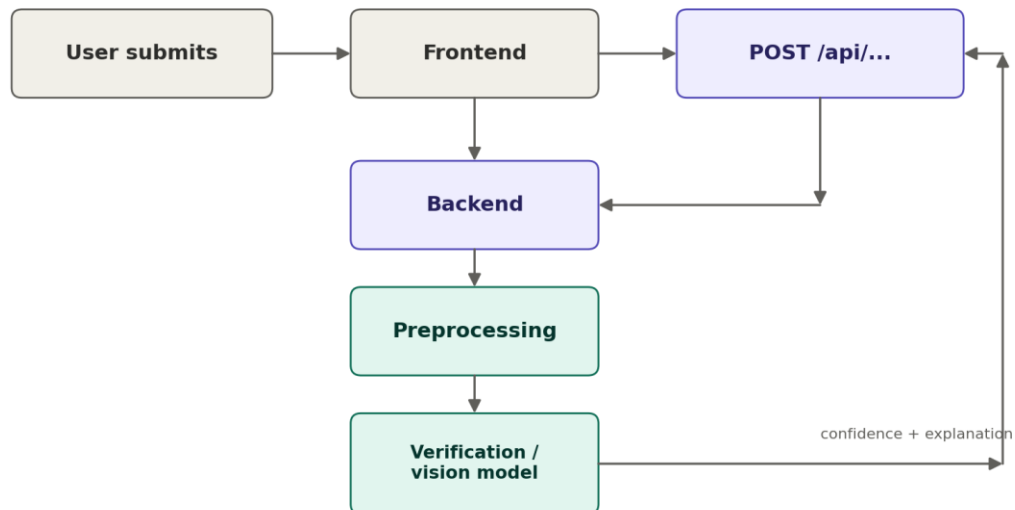


Figure 3.1: Request lifecycle from user submission to result returned to the frontend.

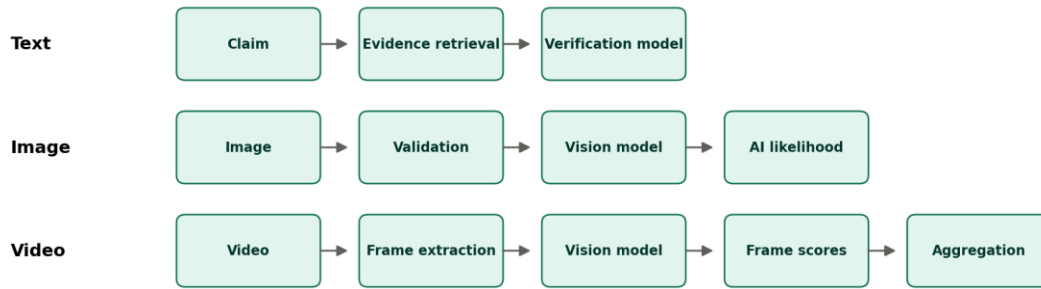


Figure 3.2: Per-type processing pipeline for text, image, and video.

## 4. Database design (ER diagram)

MVP uses SQLite. Two tables are enough: Analysis stores the result of each run, Evidence stores supporting sources for text claims. Uploaded files are not stored permanently in the MVP — only metadata and results.

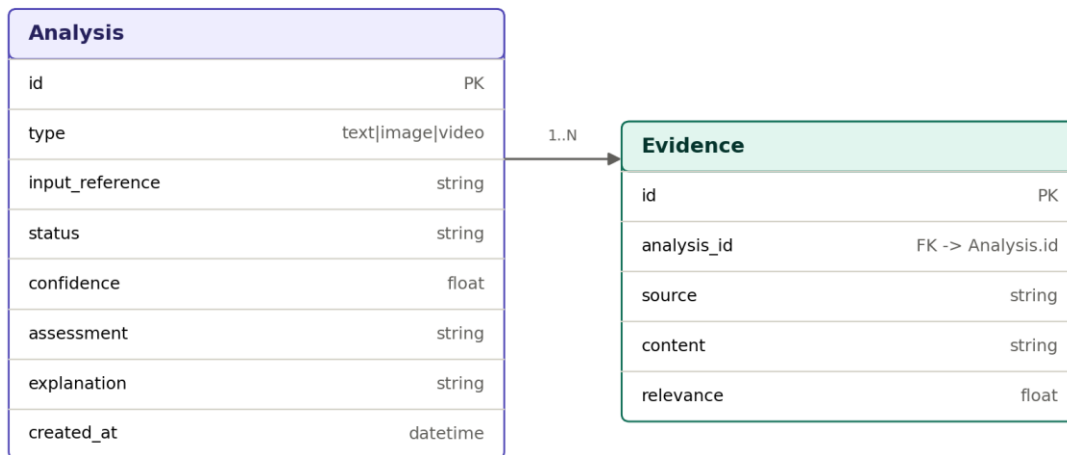


Figure 4.1: ER diagram — Analysis (1) to Evidence (many).

## 5. API contracts

### POST /api/verify-text

Request body:

```
{ "claim": "The Earth revolves around the Sun." }
```

Response body:

```
{ "assessment": "SUPPORTED", "confidence": 0.95, "evidence": [], "explanation": "..."
```

### POST /api/analyze-image



Request: multipart/form-data, field image=<file>

Response body:

```
{ "ai_likelihood": 0.87, "confidence": 0.84, "signals": [], "explanation": "..." }
```

## POST /api/analyze-video

Request: multipart/form-data, field video=<file>

Response body:

```
{ "manipulation_likelihood": 0.82, "confidence": 0.79, "signals": [], "explanation": "..." }
```

## 6. GitHub project setup

Board columns: Backlog, To do, In progress, Done.

Labels: feature, bug, documentation, frontend, backend, ml-text, ml-image, ml-video, testing, research, week-2, week-3, week-4, priority-high, priority-medium, priority-low.

### Task allocation

| Track               | Tasks                                                |
|---------------------|------------------------------------------------------|
| ML — text           | FEVER preprocessing, text baseline, evaluation       |
| ML — image          | GenImage preprocessing, image baseline, evaluation   |
| ML — video          | FF++ preprocessing, frame extraction, video baseline |
| Frontend            | Home, text UI, image UI, video UI, results screen    |
| Backend             | FastAPI setup, text/image/video endpoints            |
| Architecture (lead) | SRS, architecture, API design, integration           |

## 7. Week 2 checklist

UI: Home, text, image, video, results, and history wireframes.

Architecture: system diagram, text/image/video flowcharts, ER diagram, basic API design.

GitHub: configure project board, create labels, create issues, assign tasks, add Week 2 milestone.

Git: continue on branch 0.2 — no new branch for Week 2.